

Práctica y Trabajo Tutelado de Programación Avanzada

3º Curso – Grado en Ingeniería Informática – Ingeniería del Software
Curso académico 2025-2026

1 Introducción

La práctica y el trabajo tutelado de la asignatura “Programación Avanzada” girarán en torno al diseño e implementación de una sencilla aplicación web SPA de venta de entradas de cine. El apartado 2 especifica la funcionalidad de la práctica. El apartado 3 especifica la funcionalidad del trabajo tutelado. Finalmente, el apartado 4 contiene la normativa y evaluación de la práctica.

2 Práctica

2.1 Modelo de información

El cine dispone de un conjunto de salas. Cada **sala** tiene un nombre (e.g. Sala 1) y una determinada capacidad.

Por cada **película** se guardará el título, un resumen y la duración en minutos. Cada sala ofrece varias sesiones. Una **sesión** permite ver un pase de una película en una determinada sala en una fecha (día y hora). Cada sesión puede tener un precio distinto.

Por cada **usuario** el sitio web guarda su seudónimo (userName), contraseña, nombre, apellidos y dirección de correo electrónico.

Un usuario puede realizar una **compra** de “n” (de 1 a 10) localidades para ver una película en una de las sesiones en las que se ofrece. Para realizar la compra, el usuario debe indicar el número de localidades y un número de tarjeta bancaria.

2.2 Interacción con el usuario

El sitio web de cine distinguirá dos tipos de usuarios (roles): espectador y taquillero.

El sitio web ofrecerá las siguientes funcionalidades:

- **Registro de usuarios.** Debe permitir registrar nuevos usuarios de tipo espectador, así como permitir cambiar la información básica de registro y la contraseña para todo tipo de usuarios. Los usuarios de tipo taquillero no se registran, sino que disponen de un seudónimo y una contraseña proporcionados por la empresa del sitio web.

- **Autenticación y salida.** Un usuario (espectador o taquillero) se autenticará indicando su seudónimo y contraseña. El usuario autenticado podrá salir explícitamente del sitio web.
- **Películas en cartelera.** Un usuario, aunque no esté autenticado, podrá visualizar la cartelera. Por defecto, la cartelera muestra las películas (en orden alfabético) que se pueden ver hoy. Por cada película se mostrará su título y las horas de las sesiones disponibles (de menor a mayor) para ver la película (e.g. 18:00, 20:00, 22:00). Si una sesión ya comenzó, no se mostrará. Si una sesión empieza más allá de las 23:59, se considera que es del siguiente día. El título de la película llevará asociado un enlace que permitirá ver la información detallada de la película (título, resumen, duración y un enlace para volver a la pantalla anterior). La hora de cada sesión llevará asociado un enlace que mostrará el nombre de la película (con un enlace para ver la información detallada), la duración, el precio, día (e.g. 31-1-2023), hora (e.g. 20:00), el nombre de la sala, el número de localidades disponibles y un enlace para volver a la pantalla anterior. La cartelera también incluye un desplegable que permite seleccionar el día (hoy o cualquiera de los 6 siguientes días) de las películas que muestra la cartelera.
- **Compra de entradas.** Si un usuario de tipo espectador está autenticado y todavía quedan localidades disponibles, la información detallada de una sesión incluirá un formulario que permite seleccionar el número de entradas que desea adquirir (de 1 a 10) y el número de la tarjeta bancaria con la que va a pagar. Tras realizar la compra, el sitio web le mostrará un identificador de compra (en otra pantalla) que deberá presentar, conjuntamente con su tarjeta de bancaria, para recoger las entradas asociadas a esa compra en la taquilla en cualquier momento antes del inicio de la película.
- **Visualización del histórico de compras.** Si un usuario de tipo espectador está autenticado, la cabecera incluirá un enlace u opción (de un menú) que le permitirá visualizar todas las compras que lleva hecho a lo largo del tiempo. Las compras se mostrarán ordenadas por fecha (día y hora) de compra, mostrando primero las más recientes. Por cada compra se mostrará la fecha (día y hora) de compra, el identificador de la compra, el título de la película, el número de entradas adquiridas, el precio total, fecha (día y hora) de la sesión y una indicación de si se entregaron las entradas o no.
- **Entrega de las entradas de una compra.** Si un usuario de tipo taquillero está autenticado, la cabecera incluirá un enlace u opción (de un menú) para que pueda entregar las entradas de una compra. Tras hacer clic en dicho enlace/opción, la aplicación mostrará un formulario para que el taquillero pueda introducir el identificador de la compra y número de la tarjeta bancaria. La aplicación comprueba que la película todavía no comenzó, que la compra se hizo con esa tarjeta y que las entradas todavía no se entregaron. En caso afirmativo, registra que las entradas quedan entregadas y muestra un mensaje de éxito (en la

misma pantalla). Queda fuera del alcance de la práctica el cobro de las entradas y la impresión de los tiques.

Para no alargar innecesariamente la práctica, los datos de las salas, películas, sesiones, así como los usuarios de tipo “taquillero”, se introducirán mediante sentencias `INSERT INTO` en el script de inserción de datos (`backend/src/sql/2-MySQLCreateData.sql`).

2.3 Pruebas de integración de la capa modelo

Para verificar el correcto funcionamiento de la capa modelo, se implementarán pruebas automatizadas de los servicios de la capa modelo. **Será necesario implementar los casos de prueba más representativos (tanto de éxito como de error) de cada caso de uso.**

2.4 Internacionalización

La aplicación debe tratar la internacionalización de los mensajes, fechas y cantidades monetarias.

3 Trabajo tutelado

El objetivo del trabajo tutelado es introducir al alumno en el desarrollo de pruebas automatizadas contra la interfaz de usuario, comúnmente conocidas con el nombre “End-To-End (E2E) Testing”. En el caso de una aplicación web, en este tipo de pruebas, cada caso de prueba arranca una instancia del navegador, reproduce automáticamente los pasos que realiza el usuario final para ejecutar un caso de uso y comprueba que éste funciona correctamente, verificando que en la página que visualiza el navegador aparece la información esperada. Por ejemplo, en pa-shop, el caso de prueba correspondiente al caso de uso “login” podría constar de los siguientes pasos (a continuación, se asume que en la base de datos ya existe el usuario “user” con contraseña “secret”):

- Acceder a la URL <http://localhost:5173>.
- Hacer clic en el enlace de login.
- Introducir el seudónimo de usuario (“user”) y contraseña (“secret”) en los campos del formulario de login.
- Hacer clic en el botón de autenticación.
- Localizar el enlace asociado al seudónimo del usuario en la cabecera.
- Comprobar que el texto del enlace contiene el seudónimo del usuario (“user”).

En este trabajo tutelado se usará:

- La versión Java de Selenium WebDriver (<https://www.selenium.dev/documentation/webdriver>). Esta librería permite arrancar un navegador, cargar una página, realizar interacciones sobre ella (e.g. rellenar campos de un formulario, hacer clic en botones y enlaces, etc.), así como acceder a los elementos HTML que muestra la página. Conceptualmente, WebDriver sigue un diseño similar al de la API de JDBC. WebDriver ofrece una

API independiente del navegador y para poder usarla con un navegador concreto se necesita un “driver”, es decir, una librería que implementa la API de WebDriver para ese navegador.

- JUnit 5 como librería de desarrollo de pruebas automáticas. Cada caso de prueba (anotado con `@Test`) usará Selenium WebDriver para implementar las interacciones que realiza el usuario con el navegador y el acceso a los elementos de la página. Para realizar las aserciones, usará JUnit.

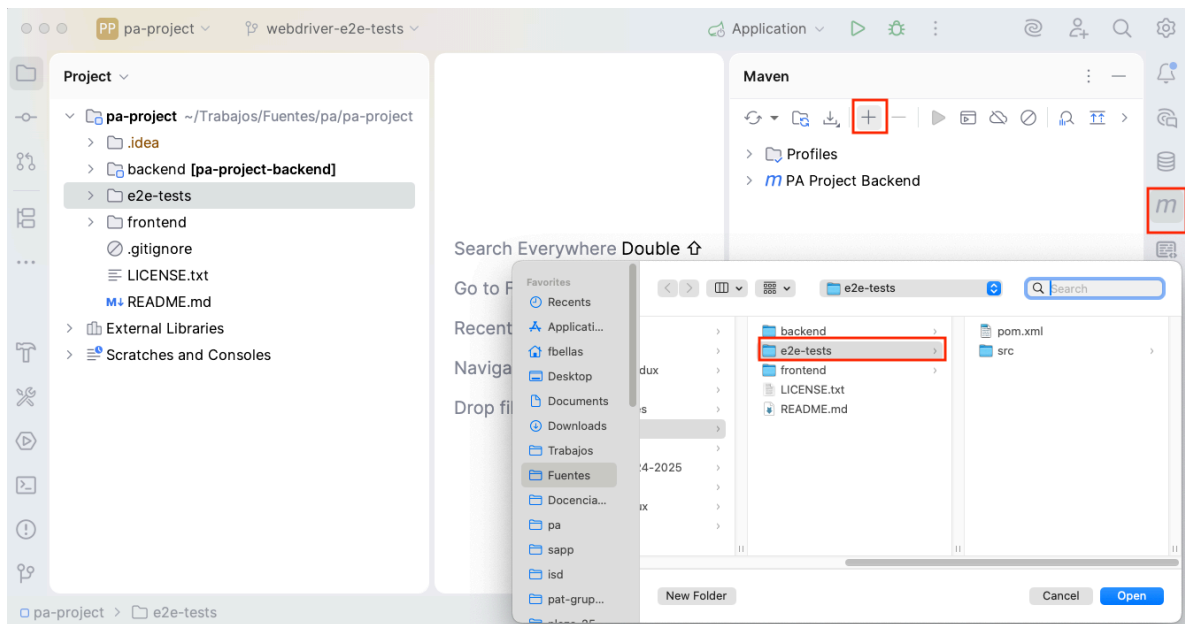
El trabajo tutelado es INDIVIDUAL y OPCIONAL. Concretamente, se proponen 4 tareas, numeradas de 0 a 3. La tarea 0 debe ser la primera en realizarse y se hará colaborativamente entre todos los miembros que deseen hacer el trabajo tutelado. Posteriormente, cada miembro que desee hacer el trabajo tutelado escogerá una de las tres tareas restantes y la completará individualmente.

Finalmente, es importante tener en cuenta que para que el trabajo tutelado se considere como válido, el código Java de las pruebas E2E NO puede generarse automáticamente con Selenium IDE (otro producto de Selenium, orientado a la creación rápida de scripts que automatizan secuencias de navegación).

3.1 Tarea 0: creación del esqueleto del proyecto de pruebas E2E

Esta tarea debe ser la primera en implementarse y su objetivo es crear la base necesaria para la implementación del resto de tareas. Consta de los siguientes pasos:

- Crear el proyecto Maven que contendrán las pruebas E2E. El proyecto debe corresponder a una carpeta hermana de las carpetas “backend” y “frontend”. Para ello, puede usarse el arquetipo maven-archetype-quickstart (<https://maven.apache.org/archetypes/maven-archetype-quickstart>) desde el directorio raíz de la práctica, introduciendo “es.udc.paproject.e2etests” como groupId, “e2e-tests” como artifactId y aceptando el resto de valores propuestos por defecto. Este comando debería crear la carpeta (proyecto Maven) “e2e-tests”, hermana de las carpetas “backend” y “frontend”.
- Agregar el proyecto Maven “e2e-tests” al proyecto de la práctica en IDEA. En la pestaña “Maven”, hacer clic en “+” y seleccionar la carpeta “e2e-tests”:



- e2e-tests/pom.xml: por defecto asume Java 17. Debe modificarse para que use Java 21.
- e2e-tests/pom.xml: incluir la dependencia de Selenium WebDriver (https://www.selenium.dev/documentation/webdriver/getting_started/install_library). Especificar una versión concreta de Selenium WebDriver 4 en lugar de `${selenium.version}`. En particular, puede usarse la versión 4.39.0 (la última disponible cuando se redactaba este documento) u otra versión 4.x.y superior si se desea. En ese último caso, puede accederse a <https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java> para averiguar la última versión 4.x.y disponible.
- e2e-tests/src/test/java/es/udc/paproject/e2etests/AppTest: hacer copy-paste del código de ejemplo que figura en https://github.com/SeleniumHQ/seleniumhq.github.io/blob/trunk/examples/java/src/test/java/dev/selenium/getting_started/UsingSeleniumTest.java. El ejemplo usa un “driver” para Chrome. Al hacer copy-paste del ejemplo en AppTest es importante observar que:
 - La clase debe seguir llamándose AppTest (en lugar de UsingSeleniumTest).
 - El paquete de la clase AppTest debe seguir siendo “es.udc.paproject.e2etests”.
 - Seguramente será necesario recargar el proyecto Maven “e2e-tests”: pestaña “m” (Maven) → e2e-tests → botón derecho → Sync Project.

- Probar a ejecutar el ejemplo:

```
cd e2e-tests
mvn test
```

- AppTest: añadir el método privado “login”, que recibe el seudónimo de un usuario (String) y su contraseña (String), y devuelve void. Implementar en este método el caso de prueba del caso de uso “login” descrito anteriormente. Este método **NO** se debe anotar con `@Test`. Para la implementación de este método es necesario tener en cuenta que:
 - Por sencillez, se usará la base de datos “papproject” para la ejecución de las pruebas de integración. Los datos necesarios para los casos de prueba se insertarán en la base de datos mediante el script `backend/src/sql/2-MySQLCreateData.sql`. Concretamente, será necesario añadir:
 - Un usuario de tipo espectador con seudónimo “testviewer” y contraseña “pa2526”.
 - Un usuario de tipo taquillero con seudónimo “testticketseller” y contraseña “pa2526”.
 - Una sesión para mañana a las 01:00 con al menos dos entradas libres.
 - Una compra del usuario “testviewer” para la sesión anterior.
 - Para saber cómo implementar cada uno de los pasos del caso de prueba debe consultarse la documentación de WebDriver (<https://www.selenium.dev/documentation/webdriver>). Un buen punto de partida puede ser leer completamente https://www.selenium.dev/documentation/webdriver/getting_started/first_script y consultar bajo demanda <https://www.selenium.dev/documentation/webdriver/elements>.
 - Se aconseja modificar el código de los componentes React para que se use el atributo “id” en los elementos HTML que se deseen localizar desde el caso de prueba (`WebElement loginLink = driver.findElement(By.id("loginLink"))`).
 - Cuando se desea obtener un elemento HTML desde un caso de prueba usando la API de WebDriver, hay que tener en cuenta que WebDriver puede no ser siempre capaz de devolverlo inmediatamente. Por ejemplo, en el caso de prueba de login, tras hacer clic en el botón de autenticarse, si se intenta acceder al enlace que contiene el userName en la cabecera (**en el código fuente este enlace ya tiene el id user-dropdown - ver Header.jsx**), es bastante probable que WebDriver no pueda devolverlo inmediatamente porque quizás el backend todavía no envió la respuesta, o si lo hizo, quizás al frontend todavía no le dio tiempo a añadir el

enlace. Existen varias estrategias para solucionar este problema. Por sencillez, se aconseja usar la estrategia “implicit wait” (<https://www.selenium.dev/documentation/webdriver/waits/#implicit-wait>).

Esta estrategia puede especificarse en el método anotado con `@BeforeEach (setup)`, y así se aplicará a todos los casos de prueba. Con esta estrategia, cuando se le pide un elemento HTML a WebDriver, la llamada correspondiente (e.g. `driver.findElement(By.id("user-dropdown"))`) se quedará bloqueada hasta que aparezca el elemento o transcurra el deadline especificado con la estrategia “implicit wait” (un deadline de 10 segundos en la práctica real será más que suficiente).

- En el método anotado con `@BeforeEach (setup)`, añadir `“driver.manage().window().maximize()”` para que el navegador se muestre maximizado. Esto garantizará que todos los elementos HTML esperados se muestren en cada pantalla. En otro caso, si la ventana del navegador fuese pequeña, Bootstrap, el framework CSS usado en el frontend, adaptaría automáticamente el HTML al tamaño de esa ventana (pequeña), y en consecuencia, no mostraría algunos elementos (que es el efecto deseado cuando se accede desde un dispositivo con pantalla pequeña).
- AppTest: renombrar el método anotado con `@Test` a “testLogin” y cambiar la implementación para que invoque a `login("testviewer", "pa2526")`.
- Ejecutar los tests:

```
cd backend
mvn sql:execute spring-boot:run
cd frontend
npm run dev
cd e2e-tests
mvn test
```

- Para depurar AppTest, basta seleccionar esta clase en la pestaña “Project”, hacer clic con el botón derecho sobre ella y elegir Debug. Como efecto colateral, IDEA crea automáticamente una configuración de ejecución que permite ejecutar y depurar dicha clase desde la barra de herramientas.

3.2 Tarea 1: prueba automatizada del caso de uso “ver información detallada de una sesión”

En esta tarea se añadirá a AppTest un caso de prueba para comprobar el funcionamiento de la visualización de la información detallada de una sesión cuando un usuario está autenticado. Concretamente, constará de los siguientes pasos:

- Autenticar al usuario “testviewer” (de tipo “espectador”), invocando al método privado `login("testviewer", "pa2526")`, desarrollado como parte de la tarea 0.
- En el desplegable del día, seleccionar la segunda opción (mañana).
- Localizar el enlace correspondiente a la primera película y guardar el texto asociado (el nombre de la película) en una variable.
- Localizar el enlace correspondiente a la primera sesión de esa película y guardar el texto asociado (la hora de la sesión) en una variable.
- Hacer clic en el enlace de la primera sesión.
- Comprobar que se incluyen todos los valores de los campos esperados (nombre de la película, duración, precio, día, hora, etc.), sin comprobar los valores concretos de cada uno. Por ejemplo, si la duración de la película se muestra con `<Card.Text id="runtime" ...>`, hay que comprobar que la página contiene un elemento HTML con ese “id”.
- Comprobar que el valor del campo correspondiente al nombre de la película coincide con el valor guardado anteriormente.
- Comprobar que el valor del campo correspondiente a la hora de la sesión coincide con el valor guardado anteriormente.
- Comprobar que incluye el formulario para comprar entradas.

3.3 Tarea 2: prueba automatizada del caso de uso “comprar entradas”

En esta tarea se añadirá a AppTest un caso de prueba para comprobar el funcionamiento de la compra de entradas. Concretamente, constará de los siguientes pasos:

- Autenticar al usuario “testviewer”, invocando al método privado `login("testviewer", "pa2526")`, desarrollado como parte de la tarea 0.
- Acceder a la URL de la información detallada de la sesión que se añadió en el script de inserción de datos en el apartado 3.1. El identificador de la sesión a la que se quiere acceder se sabrá en función del orden en el que se inserten las sesiones en el script SQL (de 1 en adelante).
- Localizar el elemento que contiene el nombre de la película y guardar el texto asociado en una variable.
- Rellenar el formulario de compra de entradas apropiadamente para poder realizar la compra de dos entradas.
- Hacer clic en el botón de comprar.
- Localizar el elemento que contiene el identificador de la compra y guardar el texto asociado en una variable.
- Hacer clic en el enlace u opción que muestra las compras que lleva hecho el usuario.
- Comprobar que el identificador de la primera compra, así como el nombre de la película asociada, coinciden con los valores esperados.

3.4 Tarea 3: prueba automatizada del caso de uso “entregar entradas”

En esta tarea se añadirá a AppTest un caso de prueba para comprobar el funcionamiento de la entrega de entradas. Concretamente, constará de los siguientes pasos:

- Autenticar al usuario “testticketseller” (de tipo “taquillero”), invocando al método privado `login("testticketseller", "pa2526")`, desarrollado como parte de la tarea 0.
- Hacer clic en el enlace u opción que permite visualizar el formulario para entregar las entradas de una compra.
- Rellenar el formulario para entregar las entradas correspondientes a la compra que se añadió en el script de inserción de datos en el apartado 3.1. El identificador de la compra se sabrá en función del orden en el que se inserten las compras en el script SQL (de 1 en adelante).
- Hacer clic en el botón de entregar.
- Comprobar que aparece un mensaje de éxito.
- Volver a rellenar el formulario con los mismos datos.
- Hacer clic en el botón de entregar.
- Comprobar que aparece un mensaje de error.

4 Normativa y evaluación

4.1 Composición de los grupos

La práctica se realizará en grupos de 3 personas.

4.2 Código fuente

El código fuente de la práctica y el trabajo tutelado deberá emplear obligatoriamente las tecnologías (y mismas versiones) estudiadas en la asignatura.

4.3 Distribución de tareas de desarrollo de la práctica

Los miembros de un grupo serán responsables de dividir el desarrollo de la práctica según consideren que es más adecuado para su caso. A modo de orientación, se recomienda que cada uno de los 3 miembros de un grupo lleve el peso mayoritario (quizás con contribuciones puntuales de otros miembros) de la implementación de 2 de las 6 funcionalidades, tanto en backend como en frontend. En todo caso, lo único que se exigirá de forma obligatoria es:

- **En cada una de las iteraciones de la práctica será necesario implementar todas las funcionalidades.**
- **Cada miembro del grupo debe contribuir de forma significativa al desarrollo de la práctica. Concretamente, cada miembro debe haber contribuido de forma relevante al desarrollo del backend y al frontend en**

cada una de sus capas. Para determinar la contribución de cada alumno a la práctica, el profesor que la corrija utilizará **EXCLUSIVAMENTE** el grafo de commits del repositorio Git asociado (en caso de dudas podrá concertar una reunión con los miembros que estime necesario).

- Los mensajes de los commits deberán seguir **OBLIGATORIAMENTE** el formato que se explica más abajo.

Los mensajes de los commits seguirán el formato “[<<etiqueta>>] Mensaje detallado”. La etiqueta identifica el caso de uso al que afecta el commit y el mensaje detallado aporta información específica. Como etiquetas se usarán las siguientes:

- FUNC-1: visualizar la cartelera.
- FUNC-2: visualizar la información detallada de una película.
- FUNC-3: visualizar la información detallada de una sesión.
- FUNC-4: comprar entradas.
- FUNC-5: visualizar el histórico de compras.
- FUNC-6: entregar las entradas de una compra.
- VARIOS: para commits de aspectos que aplican a **varios** casos de uso.

Ejemplos de commits válidos:

- [VARIOS] Añadida entidad Película.
- [VARIOS] Añadidas reglas de control de acceso en el backend.
- [VARIOS] Recuperación de la lista de salas en el backend.
- [FUNC-1] Recuperación de la cartelera en el backend.
- [FUNC-3] Visualización de la información detallada de una sesión en el frontend.
- [FUNC-3] Mejora en la visualización de la información detallada de una sesión en el frontend.

4.4 Distribución de tareas de desarrollo del trabajo tutelado

Todos los commits relativos a una tarea deben ser realizados por el alumno que haya elegido esa tarea, excepto la tarea 0. Los mensajes de los commits deben seguir el formato especificado en el apartado 4.3. Como etiquetas se emplearán:

- TT-0: Tarea 0.
- TT-1: Tarea 1.
- TT-2: Tarea 2.
- TT-3: Tarea 3.

4.5 Iteraciones y entregas

Para la realización de la aplicación se seguirá un enfoque basado en iteraciones, de manera que cada iteración incorpora más funcionalidad sobre la anterior, hasta que en la

última iteración se termina con un software que implementa toda la funcionalidad. En particular, la aplicación se hará en tres iteraciones:

- **[OBLIGATORIA] Primera iteración.** Se implementará el backend con la funcionalidad especificada en el apartado 2. **Plazo de entrega: 22 de marzo.** El profesor enviará un correo a los miembros de cada grupo con los defectos que haya identificado al analizar el código de su práctica.
- **[OBLIGATORIA] Segunda iteración.** Se corregirán los defectos detectados en la primera iteración que se estimen oportunos (**se sugiere que se corrijan todos aquellos que pueden impactar negativamente en el desarrollo de la segunda iteración**) y se implementará el frontend con la funcionalidad especificada en el apartado 2. Los alumnos que lo deseen pueden presentar también el trabajo tutelado especificado en el apartado 3. **Plazo de entrega: 17 de mayo.**
- **[OPCIONAL] Tercera iteración.** Si no se presentó el trabajo tutelado en la segunda iteración, se puede presentar en la tercera iteración. **Plazo de entrega: 7 de junio.**

4.6 Evaluación

La práctica y el trabajo tutelado se evaluarán de la siguiente manera:

- **Práctica.** Cada una de las dos primeras iteraciones se puntúan de 0 a 5, siendo la nota final de la práctica la suma de ambas. Tal y como especifican las normas de evaluación de la asignatura, se recuerda que en la evaluación de cada una de las dos iteraciones es necesario obtener un mínimo de 2,5 puntos. En caso de no alcanzarse esa nota mínima en alguna de ellas, la nota máxima de la práctica será un 4. Además, para poder presentar la segunda iteración, es necesario haber obtenido la nota mínima en la primera iteración. Dentro de un grupo, la nota puede ser diferente para cada miembro del grupo en función de su contribución y la calidad del diseño e implementación de la parte que haya hecho.
- **Trabajo tutelado.** Puntuación: de 0 a 10 puntos. En este caso no es preciso obtener ninguna nota mínima.

Para la corrección de cada iteración y el trabajo tutelado, el profesor descargará el código del repositorio Git y evaluará su correcto funcionamiento, la calidad del diseño y la calidad del código.

Una práctica copiada significará un suspenso para el grupo que ha dejado copiar y para el que ha copiado; a todos los efectos, no se hará ninguna distinción. Los suspensos por práctica copiada tendrán que realizar una práctica distinta para la segunda oportunidad, que además deberán proponer (y ser aceptada).

Si alguno de los miembros de un grupo no supera la segunda iteración (habiendo superado la primera), pero el resto sí lo hace, el alumno suspenso (o no presentado) deberá desarrollar en solitario para la segunda oportunidad una versión extendida de la práctica.

En la convocatoria correspondiente a la segunda oportunidad se presentará la misma práctica y trabajo tutelado (excepto los suspensos en alguno de los dos casos anteriores), sin posibilidad de entregar la primera iteración: se presentará directamente la versión final de la práctica y del trabajo tutelado.